

LIBIF

Hệ thống Số hóa Thư viện & Quản lý Tài liệu Thông minh

TÀI LIỆU KIẾN TRÚC HỆ THỐNG

Tài liệu thiết kế kiến trúc chi tiết cho dự án Số hóa Thư viện LIBIF. Đánh giá các phương án kiến trúc tổng thể, mô hình xử lý dữ liệu, mô hình giao tiếp và lựa chọn công nghệ (tech stack).

Trường thông tin	Nội dung
Tên dự án	LIBIF — Hệ thống Số hóa Thư viện Thông minh
Phiên bản	v1.0 (Thiết kế Kiến trúc Hệ thống)
Ngày tạo	10 tháng 07 năm 2026
Tác giả	Kiến trúc sư trưởng & Đội ngũ Kỹ thuật

1. LOẠI HÌNH ỨNG DỤNG

LIBIF là một **Ứng dụng Web Doanh nghiệp (Enterprise Web Application)** phục vụ hai luồng tương tác vận hành riêng biệt:

- Cổng thông tin Độc giả (Reader Portal):** Cổng thông tin web tương thích đa thiết bị (responsive web portal) phục vụ sinh viên/độc giả tìm kiếm và đọc sách đã được số hóa một cách an toàn.
- Trang Quản trị Thủ thư (Librarian Admin Panel):** Cổng thông tin web quản trị nội bộ dành cho thủ thư để tải lên các tệp PDF thô từ bản quét, kiểm định chất lượng, nạp dữ liệu tả (metadata) và theo dõi phân tích hệ thống.

2. ĐÁNH GIÁ & LỰA CHỌN KIẾN TRÚC TỔNG THỂ

Để thiết lập một nền tảng vững chắc, đội ngũ kỹ thuật đã đánh giá hai mô hình kiến trúc chính: **Modular Monolith** và **Microservices**:

Phương án Đánh giá	Ưu điểm cốt lõi	Thách thức & Hạn chế
Modular Monolith	• Triển khai đơn giản, chi phí vận hành thấp. • Gọi hàm nội bộ (in-process), hiệu năng cao. • Thao tác cô lập mã nguồn sạch ở cấp độ mô-đun.	• Dùng chung cơ sở dữ liệu vật lý giữa các mô-đun. • Dùng chung tài nguyên CPU/Bộ nhớ của máy chủ.
Microservices	• Mở rộng độc lập cho từng dịch vụ. • Cơ sở dữ liệu vật lý cô lập theo từng miền (domain). • Linh hoạt về công nghệ cho từng dịch vụ.	• Độ phức tạp và chi phí hạ tầng cao. • Độ trễ mạng khi gọi RPC/REST giữa các dịch vụ. • Khó khăn cho các đội ngũ kỹ thuật nhỏ.

✅ Quyết định & Lý do chọn: Modular Monolith

Hệ thống quyết định áp dụng kiến trúc **Modular Monolith**. Lựa chọn này giúp giảm thiểu độ phức tạp vận hành trong giai đoạn triển khai MVP, tối ưu hóa mức độ sử dụng tài nguyên máy chủ và rút ngắn thời gian đưa sản phẩm ra thị trường (time-to-market). Việc cô lập cấu trúc thư mục rõ ràng (Authentication, Upload, Catalog, Reader, Processing) giữ cho mã nguồn luôn sạch sẽ và tách biệt các mô-đun, sẵn sàng tách thành các vi dịch vụ (Microservices) độc lập trong tương lai nếu khối lượng công việc OCR tăng trưởng quy mô lớn.

3. LỰA CHỌN MÔ HÌNH XỬ LÝ DỮ LIỆU (PDF & OCR)

Quy trình xử lý tệp PDF (Xác thực (Validation) $\rightarrow$ Nén tệp (Compression) $\rightarrow$ Nhúng lớp văn bản tìm kiếm Tesseract OCR (vie) $\rightarrow$ Đánh chỉ mục toàn văn (Full-text Indexing)) đòi hỏi một kiến trúc dòng dữ liệu (data stream architecture) mạnh mẽ. Chúng tôi đã đánh giá hai mô hình:

Mô hình Đánh giá	Đặc điểm & Ưu điểm	Mức độ phù hợp với Số hóa Thư viện
Pipe and Filter	• Chia nhỏ quy trình thành các bộ lọc (filter) độc lập. • Dữ liệu chảy liên tục qua các bộ lọc.	✅ Cực kỳ phù hợp cho xử lý PDF thô. ✅ Dễ dàng kiểm tra chất lượng ở từng bước.
Batch Sequential	• Xử lý các lô dữ liệu lớn theo trình tự. • Mỗi bước phải chờ bước trước hoàn thành 100%.	❌ Độ trễ cao đối với các cuốn sách dày. ❌ Nguy cơ cạn kiệt bộ nhớ (OOM) khi xử lý các tệp dung lượng lớn.

✅ Quyết định: Pipe and Filter kết hợp Hàng chờ Tác vụ (Task Queuing)

Hệ thống lựa chọn mẫu thiết kế **Pipe and Filter**. Mỗi tệp PDF tải lên sẽ trải qua quy trình lọc 4 giai đoạn. Để thực hiện điều này mà không gây nghẽn tài nguyên, chúng tôi sử dụng **Redis + BullMQ** làm

hàng chờ tác vụ bất đồng bộ (asynchronous task queue). Mỗi bộ lọc chạy như một bước worker độc lập, duy trì độ ổn định cao và tối ưu hóa hiệu suất CPU.

[1. Xác thực] → [2. Nén tệp] → [3. Lớp văn bản Tesseract OCR] → [4. Đánh chỉ mục]
(Kiểm tra định dạng) (Tối ưu hóa DPI) (Lớp chữ ẩn Tiếng Việt vie) (Chỉ mục toàn văn)

4. MÔ HÌNH GIAO TIẾP VÀ TƯƠNG TÁC

Đối với việc giao tiếp giữa các mô-đun nghiệp vụ (domain modules) (ví dụ: sự kiện tải lên, thông báo hoàn tất OCR, cập nhật chỉ mục), chúng tôi đã đánh giá hai mô hình:

Mô hình	Mô tả & Ưu điểm	Thách thức
Request - Response	- Phản hồi ngay lập tức cho phía client. - Lập trình REST API đơn giản, trực quan.	- Gây nghẽn (blocking) khi xử lý các tác vụ kéo dài. - Không phù hợp cho các tác vụ nền nặng.
Event-Driven	- Phân tách lỏng lẻo (loose coupling) giữa các mô-đun nghiệp vụ. - Tách rời thời gian thực thi đối với các tác vụ nặng.	- Tăng độ phức tạp của kiến trúc. - Khó truy vết luồng dữ liệu trực quan.

✓ Quyết định: Mô hình Giao tiếp Lai (Hybrid Communication Model)

Hệ thống áp dụng mô hình **Lai (Hybrid)**:

- Request-Response (REST API qua HTTP)** cho các tương tác đồng bộ của độc giả/thủ thư (xác thực, tìm kiếm, đọc trực tuyến).
- Kiến trúc Hướng sự kiện (Event-Driven Architecture với Asynchronous Events)** cho công việc xử lý nền.

Khi thủ thư tải lên một cuốn sách, hệ thống phát ra sự kiện `BookUploadedEvent` và trả về phản hồi tức thì `202 Accepted`. Quy trình OCR (OCR Pipeline) được kích hoạt bất đồng bộ bởi Bộ xử lý sự kiện (Event Handler) mà không làm tắc nghẽn các kết nối HTTP.

Mô-đun Upload → BookUploadedEvent → Event Bus (EventEmitter) → Hàng chờ OCR Worker
(Kích hoạt Tác vụ Nền)

5. CÁC THÀNH PHẦN HỆ THỐNG

Ứng dụng được cấu trúc thành các ranh giới miền mô-đun bên trong kiến trúc **Modular Monolith**:

- **Mô-đun Xác thực (Auth Module):** Quản lý xác thực JWT, phân quyền và các vai trò RBAC (Thư thư, Độc giả, Quản trị viên).
- **Mô-đun Tải lên (Upload Module):** Tiếp nhận tệp PDF thô, lưu trữ vào Object Storage và phát các sự kiện xử lý nền.
- **Mô-đun Danh mục (Catalog Module):** Đánh chỉ mục dữ liệu tả (metadata), cung cấp bộ lọc nâng cao và tích hợp với Google Books ISBN API.
- **Mô-đun Độc giả (Reader Module):** Kiểm tra quyền truy cập, tạo presigned URL tạm thời trên S3 và hiển thị sách an toàn qua HTML5 Canvas.
- **Mô-đun Xử lý (Processing Module):** Điều phối quy trình Pipe and Filter cho công đoạn nén PDF nền và thực thi Tesseract OCR (vie).

5.1 Chi tiết Công nghệ Sử dụng (Tech Stack)

Tầng (Layer)	Công nghệ	Lý do Lựa chọn
Frontend	Next.js 14 (App Router), TailwindCSS, PDF.js	Hiệu năng cao, render phía server (SSR), giao diện tương thích responsive, trình xem Canvas DRM.
Backend	NestJS (TypeScript)	Hỗ trợ TypeScript mạnh mẽ, kiến trúc mô-đun chuẩn mực, cấu trúc quy mô doanh nghiệp.
Database	PostgreSQL 16 (Prisma ORM)	Tính toàn vẹn dữ liệu quan hệ, khả năng tìm kiếm toàn văn Tiếng Việt bản địa với tsvector.
Hàng chờ Tác vụ (Task Queue)	Redis 7 + BullMQ	Xử lý hàng chờ tin cậy, kiểm soát đồng thời (concurrency), cơ chế thử lại (retry), theo dõi tiến độ.
AI / OCR	Tesseract OCR (vie), PyMuPDF	Công cụ OCR mã nguồn mở tiêu chuẩn công nghiệp với dữ liệu huấn luyện Tiếng Việt (vie) & tiền xử lý ảnh.
Lưu trữ (Storage)	MinIO / AWS S3	Lưu trữ đối tượng (Object Storage) tiêu chuẩn cho tài sản tài liệu PDF thô và đã qua xử lý.

6. AN TOÀN DỮ LIỆU VÀ THUỘC TÍNH CHẤT LƯỢNG KIẾN TRÚC

6.1 Cơ chế An ninh & Chống Tái xuống DRM

Để bảo vệ bản quyền giáo trình thư viện, hệ thống áp dụng khuôn khổ an ninh nghiêm ngặt:

- **URL Ký trước (Presigned URLs):** Các tệp PDF vật lý trên MinIO/S3 không bao giờ bị công khai trực tiếp. Backend tạo ra các liên kết mã hóa tạm thời với **thời gian sống ngắn (TTL < 60s)**.
- **Chặn Tái xuống Trực tiếp:** Trình xem trực tuyến hiển thị các trang sách một cách động thông qua phần tử HTML5 Canvas thay vì nhúng iframe hoặc tệp PDF thô. Menu chuột phải, phím tắt F12 DevTools, bôi đen văn bản và lệnh in đều bị chặn và triệt tiêu.

6.2 Các Thuộc tính Chất lượng Kiến trúc Cốt lõi

- **Khả năng Bảo trì (Maintainability):** Ranh giới mô-đun rõ ràng bên trong kiến trúc Modular Monolith cho phép cập nhật từng miền nghiệp vụ độc lập. Ví dụ, việc thay thế thư viện OCR nội bộ bằng một nhà cung cấp đám mây không đòi hỏi bất kỳ thay đổi nào ở phía frontend.
- **Khả năng Mở rộng (Scalability):** Thiết kế sẵn sàng tách biệt. Việc sử dụng Redis + BullMQ giúp tách rời máy chủ xử lý nền khỏi thread pool của web server.
- **Khả năng Chịu lỗi (Fault Tolerance):** BullMQ cung cấp chính sách thử lại tự động (3 lần thử lại) đối với các sự cố mạng tạm thời hoặc lỗi xử lý tệp.

7. KẾT LUẬN KIẾN TRÚC

Kiến trúc cho Hệ thống Số hóa Thư viện LIBIF được tối ưu hóa về an toàn thông tin, hiệu năng và chi phí vận hành:

- Áp dụng kiến trúc **Modular Monolith** để tinh gọn giai đoạn phát triển ban đầu và giảm thiểu chi phí hạ tầng.
- Áp dụng mô hình **Pipe and Filter** được quản lý bởi hàng chờ worker **Redis + BullMQ** cho tác vụ OCR bất đồng bộ nặng.
- Áp dụng **Mô hình Giao tiếp Lai (Hybrid Communication)** (REST và Hướng sự kiện) cho tương tác người dùng phản hồi nhanh chóng.

Thiết kế này mang lại mức độ an toàn cao, hiệu năng vượt trội và khả năng mở rộng rõ ràng cho quá trình chuyển đổi số thư viện.